LQR as a Nav2 Controller Plugin

for TurtleBot3 Path Tracking

Tommy Oh Ansh Aggarwal Daniel Senteu JunHang Wu

CMPT 720: Robot Motion Planning and Control · Simon Fraser University · Spring 2026

100%	67.9s	0.083m	6
Goal Success Rate	Fastest Time-to-Goal	Mean CTE (LQR+NavFn)	Bugs Fixed Before Tuning

OVERVIEW

Contents

01
Motivation & Background
Why LQR for Nav2? What gap does it fill?

02
Robot Model & Problem Setup
Unicycle dynamics, body-frame error, cost formulation

03
LQR Algorithm
Linearization, DARE, gain computation, control law

04
Implementation & Bug Fixes
6 critical bugs discovered and resolved

05
Benchmark Results
3 controllers × 2 planners, all metrics compared

06
Discussion & Conclusion
Key takeaways, limitations, future work

Why LQR for Nav2?

The Gap

- Nav2 ships with DWB (heuristic local planner) and MPPI (sampling-based). Neither is analytically optimal.
- DWB uses dynamic window sampling — no formal stability guarantee.
- MPPI requires many stochastic rollouts — computationally intensive and produces noisy commands.
- LQR: closed-form optimal gain from a quadratic cost, computed once — runs at full 20 Hz with minimal overhead.

Course Connections

A1 Q2 — **Dubins car model**
Same unicycle dynamics used here

A1 Q3 — **MCL Localization**
Nav2 uses AMCL throughout our stack

A2 Q3 — **MPPI Controller**
Our primary stochastic baseline

A3 Q1 — **LQR Trajectory Tracking**
Directly extends course LQR to Nav2

Unicycle Dynamics & Body-Frame Error

Continuous-Time Unicycle Model

$$\dot{x} = v \cos\theta$$
$$\dot{y} = v \sin\theta$$
$$\dot{\theta} = \omega$$

State: $x = [p_x, p_y, \theta]^T$ Control: $u = [v, \omega]^T$

Body-Frame Tracking Error

$$e_{long} = \cos\theta_r \, \Delta x + \sin\theta_r \, \Delta y$$
$$e_{lat} = -\sin\theta_r \, \Delta x + \cos\theta_r \, \Delta y$$
$$e_{\theta} = \mathrm{normalize}(\theta - \theta_r)$$

Error vector: $e = [e_{long}, e_{lat}, e_{\theta}]^T \in \mathbb{R}^3$

Discrete Linearization (Forward Euler, $v_{ref} = 0.2$ m/s, $\Delta t = 0.05$ s)

$$A_d = I + A_c \cdot \Delta t = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0.01 \\ 0 & 0 & 1 \end{bmatrix}$$

$$B_d = B_c \cdot \Delta t = \begin{bmatrix} 0.05 & 0 \\ 0 & 0 \\ 0 & 0.05 \end{bmatrix}$$

Key: Since v_{ref} and $\omega_{ref} = 0$ are constant, A_d and B_d are constant → DARE solved once at startup.

DARE Solver & Gain Computation

Discrete Algebraic Riccati Equation (Iterative)

$$P_{k+1} = Q + A^T P_k A - A^T P_k B (R + B^T P_k B)^{-1} B^T P_k A$$

Convergence: $\max|P_{k+1} - P_k| < 10^{-9}$ (typically < 100 iterations)

LQR Gain $K \in \mathbb{R}^{2\times3}$

$$K = (R + B^T P B)^{-1} B^T P A$$

Computed once after DARE converges

Control Law

$$\delta u = -K \cdot e$$
$$v = v_{ref} + \delta u_0, \quad \omega = \delta u_1$$

Clamped: $v \in [0, 0.22]$ m/s $|\omega| \leq 2.84$ rad/s

Final Cost Matrices (no tuning required — original spec gains passed all exit criteria)

$Q = \mathrm{diag}(1.0, 3.0, 1.0)$ $R = \mathrm{diag}(1.0, 0.5)$ q_lateral = 3.0 elevated to prioritize cross-track accuracy

Nav2 Plugin Architecture

NavFn / Smac 2D Planner

↓ nav_msgs/Path

Controller Server (20 Hz)

↓ computeVelocityCommands()

LQR Controller Plugin

↓ TwistStamped

/cmd_vel → TurtleBot3

lqr_solver: pure Eigen math | lqr_controller: Nav2 plugin (ROS2)

Control Loop — Per Cycle (20 Hz)

1. **findClosestPoint(pose)**
   Forward search from last index — prevents backward snap
2. **findLookaheadPoint(idx)**
   Advance 0.5 m along path for reference pose
3. **computePathHeading(ref)**
   atan2(dy,dx) fallback for NavFn identity quaternions
4. **computeBodyFrameError()**
   Compute $e = [e_{long}, e_{lat}, e_\theta]^T$
5. **$\delta u = -K \cdot e$**
   Apply precomputed LQR gain
6. **clamp(v, ω)**
   Enforce TurtleBot3 hardware limits

Six Bugs Before Any Q/R Tuning

The robot never reached the goal — not because of bad gains, but because of 6 code bugs. All were fixed before touching Q or R.

CRITICAL

Backward Path Snap

Full-path scan → index snapped backward near goal → robot drove away from goal

CRITICAL

Goal Overshoot & Oscillation

Constant vref = 0.2 m/s → overshot 0.25 m tolerance → oscillated indefinitely

CRITICAL

Goal Checker Never Called

goal_checker parameter silently discarded → controller kept commanding velocity

MODERATE

Wrong Heading from NavFn

NavFn outputs identity quaternion (yaw=0 everywhere) → body-frame error corrupted

MODERATE

Progress Checker False Abort

0.5 m / 10 s threshold → Nav2 aborted during fine approach near goal

MINOR

size_t Underflow

poses.size()-1 on 1-element path → wraps to 2^{64}, undefined behavior

After fixes: 5/5 runs reached goal · Mean CTE 0.136 m · No Q/R tuning ever performed

No Gains Were Changed

2026-03-23

Exp 0 — Baseline — Q = diag(1,3,1), R = diag(1,0.5)

AGENT.md spec defaults. Mean CTE 0.10 m — tracking fine in plots, but goal-reaching unreliable. Commit: 9e3a5bd.

Goal-reaching fails

2026-03-27

Exp 1 — 6 Bug Fixes — Same Gains

Six controller bugs fixed (see previous slide). First post-fix run: "[controller_server]: Reached the goal!" confirmed in logs. Commits: cc55b0a, 1d4a366, f32b14e.

Goal reached ✓

2026-04-03

Exp 2 — 5-Run Validation — Phase 4 Exit Criteria Met

Benchmark script also fixed (was reporting ~2.3 m CTE from wrong trajectory source). Final: 5/5 success, mean CTE 0.136 m — below 0.15 m target.

100% success ✓

Key finding: The robot's failure to reach the goal was entirely due to code bugs — not suboptimal Q/R gains.

Benchmark Setup

3 Controllers × 2 Planners = 6 Configurations

	NavFn	Smac 2D
LQR	LQR + NavFn	LQR + Smac
DWB	DWB + NavFn	DWB + Smac
MPPI	MPPI + NavFn	MPPI + Smac

Metrics Collected

- Mean & Max Cross-Track Error (CTE)
- Smoothness: RMS dv/dt and dω/dt

Test Conditions

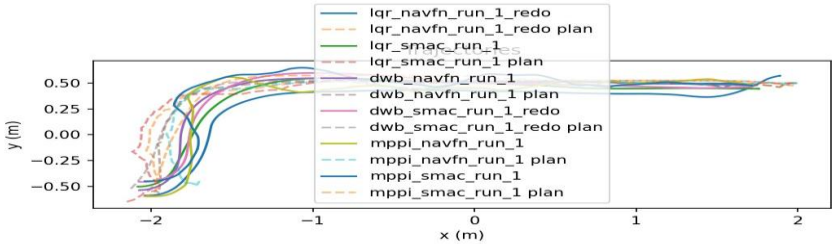
- World: turtlebot3_world (Gazebo)
- Start: default spawn position

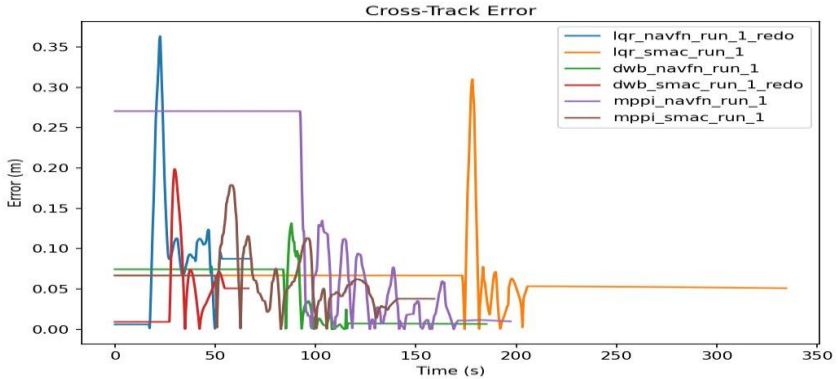
6-Combo Benchmark Results

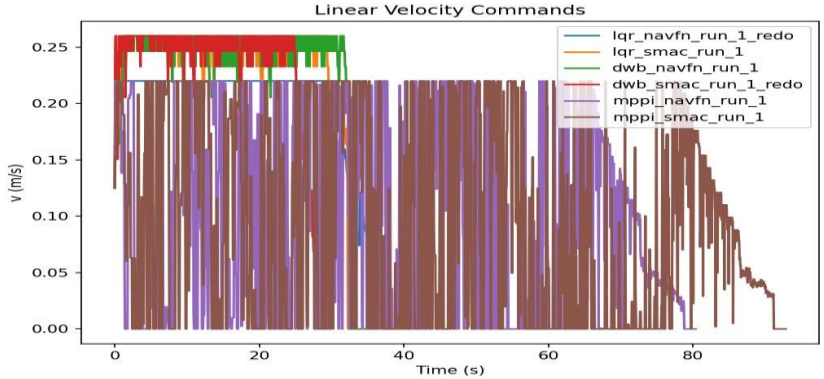
Configuration	Mean CTE	Max CTE	dv/dt	dω/dt	Time (s)	Goal
LQR + NavFn	0.083 m	0.363 m	**0.110**	0.939	**67.9 s**	✓
LQR + Smac 2D	0.061 m	0.310 m	—	—	334.6 s	✓
DWB + NavFn	**0.042 m**	**0.131 m**	0.219	**0.876**	185.1 s	✓
DWB + Smac 2D	**0.042 m**	0.199 m	0.277	0.886	**66.4 s**	✓
MPPI + NavFn	0.146 m	0.270 m	1.088	2.683	197.2 s	✓
MPPI + Smac 2D	0.060 m	0.179 m	1.029	—	159.1 s	✓

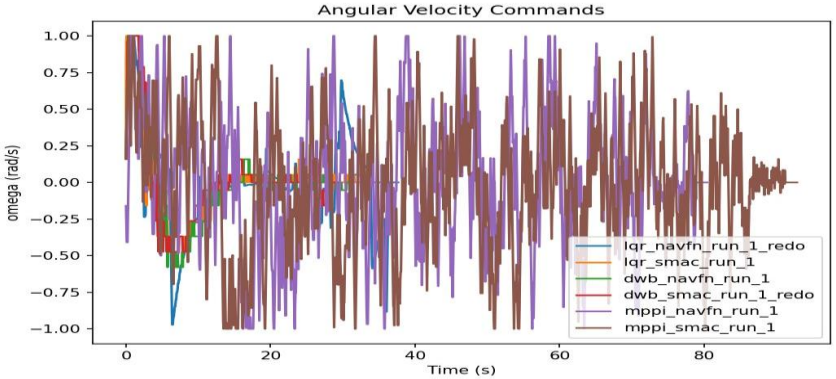
★ = per-column best | — = excluded (timestamp artifacts in rosbag recording) | Orange rows = LQR

Benchmark Comparison Plots



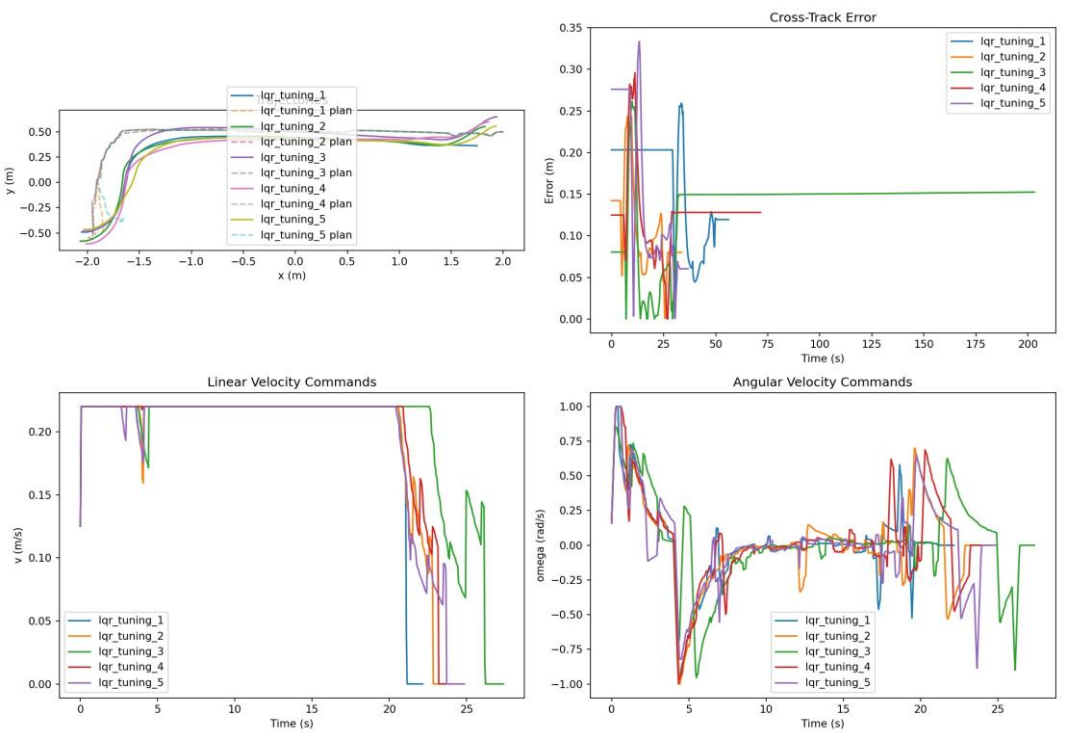






Trajectory overlay (top-left) · CTE over time (top-right) · Linear velocity v(t) (bottom-left) · Angular velocity ω(t) (bottom-right)

5-Run LQR Reliability Validation



Validation Summary

Success Rate

5 / 5

100%

Mean CTE

0.136 m

target < 0.15 m ✓

Max CTE range

0.24–0.33 m

across 5 runs

Smoothness dv/dt

~0.15

consistent, no oscillation

Smoothness dω/dt

~1.03

consistent, no oscillation

Q / R tuning

None

original spec gains kept

Same gains · same world · same goal across all 5 trials

Analysis & Key Takeaways

LQR vs DWB

DWB achieves lower CTE (0.042 vs 0.083 m) but LQR+NavFn is 2.7× faster (67.9 vs 185.1 s). Smoothness is comparable. DWB+Smac is closest competitor on both metrics.

LQR vs MPPI

LQR dominates on every metric. MPPI+NavFn: worst CTE (0.146 m), highest dω/dt (2.68), 1609 cmd_vel messages vs 586 for LQR. Sampling-based corrections compound into jitter.

Bugs Before Tuning

Most important lesson: correct implementation matters more than gain selection. 6 bugs prevented goal-reaching; fixing them — not tuning — delivered 100% success.

Limitations

Single run per combo (no error bars). Simulation only — real robot requires re-tuning for wheel slip and LiDAR noise. Fixed start/goal pair may not generalize.

Conclusion

01

Implemented LQR as a Nav2 Plugin

C++ Nav2 controller plugin with DARE-based steady-state gain, running at 20 Hz on a TurtleBot3 Burger in Gazebo.

02

Bugs Matter More Than Gains

All 6 implementation bugs were fixed before Q/R tuning — the original spec gains (Q=diag(1,3,1), R=diag(1,0.5)) achieved 100% success rate.

03

Competitive Performance vs DWB/MPPI

LQR: fastest time-to-goal (67.9 s), smoothest commands (dv/dt 0.11), and consistent 5-run validation at mean CTE 0.136 m.

04

Future Work

Hardware deployment on physical TurtleBot3 · Multi-run statistical benchmarking · Adaptive LQR with gain scheduling · Safety filter integration (CBF / HJ Reachability).